CHRONO SUPPORT FOR HUMAN-IN-THE-LOOP SIMULATION

Speaker: Kyle Sha
(Simulation-Based Engineering Lab)

Two patterns, one contract

- Human-in-the-loop = the simulation runs at wall-clock speed and a person closes the control loop
- Two things here are worth your time, because they transfer to anything outside Chrono talking to a live sim:
  - PATTERN 1: keeping a simulation real-time, and measuring how well you did
  - PATTERN 2: taking input from a device Chrono has never heard of, and closing the loop back to it
- Between them, the easy case: a human on the keyboard, which Chrono already does for you
- The contract joining all of it is three floats per step, in, and some state, out
- Everything runs in PyChrono; no special hardware is required
  - the rest of the repo - gamepad, gears, Mcity, a rover, a crane - is one slide at the end and a README

Before we start ……

- This is a walkthrough - nothing to install right now; everything runs on my machine
- Code, slides and a written walkthrough: github.com/ksha23/chrono-hil-tutorial
- To run it yourself afterward:
  - `conda create -n chrono_hil -c projectchrono -c conda-forge pychrono pygame python=3.13`
  - `conda activate chrono_hil`
- Do NOT `pip install pychrono` - the PyPI package with that name is an unrelated library
- `python tutorial_HIL_driver.py` opens an Irrlicht window with an HMMWV - drive it with the arrow keys
- All switches are in the `CONFIGURATION` section at the bottom of `tutorial_HIL_driver.py`

What Chrono actually requires of a human interface (Lines 151 to 178)

```
class DriverInputs:
    """The whole ChDriver contract, in Python."""

    def __init__(self):
        self.m_steering = 0.0
        self.m_throttle = 0.0
        self.m_braking = 0.0

    def SetSteering(self, v):
        self.m_steering = v

    def SetThrottle(self, v):
        self.m_throttle = v

    def SetBraking(self, v):
        self.m_braking = v

    def GetInputs(self):
        return self
```

Three floats per step: steering [-1, 1], throttle [0, 1], braking [0, 1]

`veh.ChDriver` is the same container in C++; this Python stand-in is what the rover and the crane use

That is the entire contract. Everything else in this talk is about filling it in, on time, from somewhere real

Pattern 1: keep the simulation real-time

What "real-time" means in Chrono

- Chrono integrates as fast as it can; it has no notion of wall-clock time by itself
- Real Time Factor: RTF = compute time for a step / step size
  - RTF < 1: faster than real time - we must WAIT after each step
  - RTF > 1: slower than real time - nothing can be done except a cheaper model or a bigger step
- Soft real-time: spin after each step until wall time catches up with simulation time
  - No OS scheduling guarantees; good enough for a driving simulator, not for a control ECU
- Three equivalent mechanisms in PyChrono: `ChRealtimeStepTimer`, `ChVehicle.EnableRealtime`, or your own

The loop, and where the driver is queried (Lines 674 to 725)

```
while vis.Run():
    sim_time = system.GetChTime()

    # Render scene
    if step_number % render_steps == 0:
        follow_camera()
        vis.BeginScene()
        vis.Render()
        vis.EndScene()

    # PART 4 samples the device here; PART 6 applies any gear command
    ...

    # Get driver inputs (three floats) - the whole human-in-the-loop contract
    driver_inputs = driver.GetInputs()

    # PART 8: hand the three numbers to whatever is being controlled
    plant.apply(driver_inputs)

    # Update modules (process inputs from other modules)
    driver.Synchronize(sim_time)
    terrain.Synchronize(sim_time)
    plant.synchronize(sim_time, driver_inputs)
    vis.Synchronize(sim_time, driver_inputs)

    # Advance simulation for one timestep for all modules
    driver.Advance(step_size)
    terrain.Advance(step_size)
    plant.advance(step_size)          # spins here if REALTIME == "vehicle"
    system.DoStepDynamics(step_size)  # only for a plant that does not step itself
    vis.Advance(step_size)
```

Synchronize: every module reads what it needs from the others at time t. Advance: every module integrates t to t + step
The driver is queried BEFORE Synchronize - the human's input is sampled once and held for the whole step

Measure it: sim time vs wall time (Lines 727 to 743)

```
# Console report: how far is the sim from wall time?
if step_number % report_steps == 0:
    wall = time.perf_counter() - wall_start
    # A ChVehicle measures its own real-time factor per step. The rover
    # and the crane do not, so it is measured here over the reporting
    # interval instead: same quantity, coarser sampling.
    if vehicle:
        rtf = vehicle.GetRTF()
    else:
        d_wall = wall - last_wall
        d_sim = sim_time - last_sim
        rtf = (d_wall / d_sim) if d_sim > 0 else 0.0
        last_wall, last_sim = wall, sim_time
    print(f"{sim_time:7.2f} {wall:7.2f} {wall - sim_time:+7.3f} {rtf:6.2f}"
          f" {plant.speed():7.2f}  {driver_inputs.m_steering:6.2f}"
          f" {driver_inputs.m_throttle:5.2f} {driver_inputs.m_braking:5.2f}"
          f"  {plant.status()}")
```

drift = wall time - simulation time: negative means the sim is ahead of the clock

`vehicle.GetRTF()` is the true RTF of the last step, even when real time is being enforced

Measure before you fix. Most of Pattern 1 is knowing the number, not enforcing it

Show Time

- `REALTIME = "none"`: watch the drift column run away
- Change `step_size` to 5e-4 - RTF goes above 1 and the sim can never be real-time, whatever the timer does

Enforce real time, three ways (Lines 631 to 645, 71 to 88)

```
# Real-time setup (PART 1)
realtime_mode = REALTIME
if REALTIME == "vehicle":
    if vehicle:
        vehicle.EnableRealtime(True)
    else:
        # PART 8: there is no vehicle to do the spinning, so fall back to
        # the timer that does the same thing from outside.
        realtime_mode = "per_step"
rt_timer = chrono.ChRealtimeStepTimer()  # used when REALTIME == "per_step"
cum_timer = CumulativeRealtimeTimer()    # used when REALTIME == "cumulative"

# ... and the third option, defined at the top of the file:

class CumulativeRealtimeTimer:
    """Soft real-time that does not accumulate drift.

    ChRealtimeStepTimer measures each step independently: any time lost on a
    slow step (or spent in Python between steps) is never recovered.  This timer
    instead compares the *total* simulated time with the *total* wall time, so a
    slow step is followed by a few fast steps that catch back up.
    """

    def reset(self):
        self.t_start = time.perf_counter()

    def spin(self, sim_time):
        while time.perf_counter() - self.t_start < sim_time:
            pass  # spin in place until real time catches up
```

"vehicle" spins inside `vehicle.Advance()`; "per_step" is the same timer called explicitly; "cumulative" is ours
Per-step timers never recover time lost on a slow step. Comparing TOTAL sim time to TOTAL wall time does

Show Time

- Measured on a laptop, HMMWV + TMeasy tires, step 3 ms (RTF ~ 0.14):
  - "none": drift -69 s after 17 s of wall time
  - "per_step": drift +0.012 s after 17 s and growing
  - "vehicle": drift +0.021 s after 17 s and growing
  - "cumulative": drift +0.000 s
- Drift matters more the closer you are to RTF = 1

The easy case: a human on the keyboard

ChInteractiveDriver, and routing events (Lines 535 to 546, 589 to 592)

```
elif INPUT_SOURCE == "keyboard":
    ### PART 2: keyboard through the Irrlicht window ###
    # Arrow keys steer/accelerate/brake, 'C' centers steering, 'R' releases
    # the pedals, 'L' locks the current inputs.
    driver = veh.ChInteractiveDriver(vehicle)
    steering_time = 1.0  # time to go from 0 to +1 (or from 0 to -1)
    throttle_time = 1.0  # time to go from 0 to +1
    braking_time = 0.3   # time to go from 0 to +1
    driver.SetSteeringDelta(render_step_size / steering_time)
    driver.SetThrottleDelta(render_step_size / throttle_time)
    driver.SetBrakingDelta(render_step_size / braking_time)
    driver.SetGains(4.0, 4.0, 4.0)  # first-order lag from key target to applied input

# ... and then, when the Irrlicht window is built:

vis.AddSkyBox()
plant.attach(vis)
if INPUT_SOURCE in ("keyboard", "gamepad"):
    vis.AttachDriver(driver)  # route Irrlicht key/joystick events to the driver
```

Each keypress moves a TARGET by "delta"; the applied input follows it with a first-order lag (`SetGains`)

`AttachDriver` forwards the window's key events - and binds Chrono's gear keys `Z X T [ ]` for free

Nothing else in the loop changes: the driver still just returns three floats

Show Time

- Driving the HMMWV with the arrow keys - watch the Irrlicht HUD (speed, inputs, RTF)
- Everything Chrono knows how to read, it reads for you. The rest of the talk is what happens when it doesn't

Pattern 2: a device Chrono doesn't know about

a UDP console, a phone, a motion platform, another process, another machine…

Two processes, one UDP contract

Both directions are plain UDP datagrams - two processes, possibly two machines, no shared memory. The return arrow only fires when SEND_FEEDBACK = True.

A UDP input source (Lines 219 to 269)

```
class UdpInput:
    """Receive 'steering,throttle,braking[,gear]' datagrams from an operator.
    Non-blocking: the physics loop never waits.  No packet since the last step
    means the previous target is held (zero-order hold)."""

    def __init__(self, port=9870):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(("0.0.0.0", port))
        self.sock.setblocking(False)
        self.operator_addr = None
        self.last = (0.0, 0.0, 0.0)
        self.gear_commands = []

    def poll(self):
        # Drain everything queued and keep only the newest packet.  Gear commands
        # are the exception: they accumulate, because dropping one loses a shift.
        while True:
            try:
                data, addr = self.sock.recvfrom(256)
            except BlockingIOError:
                break
            fields = data.decode().split(",")
            if len(fields) < 3:
                continue
            try:
                s, t, b = (float(x) for x in fields[:3])
            except ValueError:
                continue
            self.last = (s, t, b)
            self.operator_addr = addr
            if len(fields) > 3 and fields[3] and fields[3] != "-":   # PART 6
                self.gear_commands.append(fields[3][0])
        return self.last
```

Non-blocking socket, drained every step: the newest packet wins, and an empty queue holds the last value
The operator can be anywhere on the network, and the console has no Chrono dependency at all

Sample once, hold, then send state back (Lines 684 to 696, 745 to 755)

```
# PART 4: sample the device ONCE per step and hold it for the step
if device is not None:
    smoother.set_target(*device.poll())
    smoother.advance(step_size)
    smoother.apply_to(driver)
    # PART 6: gear commands are events, not levels, so they are applied
    # once each rather than held like the three numbers above.
    if gearbox is not None:
        for command in device.take_gear_commands():
            gearbox.command_char(command)

# Get driver inputs (three floats) - this is the whole human-in-the-loop contract
driver_inputs = driver.GetInputs()

# ... and at the end of the same step:

# PART 4: feedback to the operator (speed, RTF, lateral acceleration, applied
# inputs, and since PART 6 the gear)
if SEND_FEEDBACK and device is not None and step_number % render_steps == 0:
    acc = (vehicle.GetPointAcceleration(chrono.ChVector3d(0, 0, 0)).y
           if vehicle else 0.0)
    rtf = vehicle.GetRTF() if vehicle else 0.0
    gear = gearbox.describe() if gearbox else plant.status() or "--"
    device.send_feedback(
        f"{sim_time:.3f},{plant.speed():.3f},{rtf:.3f},{acc:.3f},"
        f"{driver_inputs.m_steering:.3f},{driver_inputs.m_throttle:.3f},"
        f"{driver_inputs.m_braking:.3f},{gear}")
```

poll -> smooth -> write, then `GetInputs()` as before; the physics loop never waits on the device

Feedback goes back at render rate (50 Hz), not every physics step - the human cannot use 333 Hz

Show Time

- Two terminals on one laptop over localhost - or two machines over the network, same code either way
- `SEND_FEEDBACK = True`: speed, RTF and lateral acceleration come back to the console while you drive
- Stop the console mid-drive: the last input is held - what should a real simulator do here?

How far this goes: gear, ground, and plant

- PART 6 - change gear. A gear is not a fourth float: it belongs to the vehicle, not to the driver
  - the keyboard binds `Z X T [ ]` for free once `AttachDriver` is called; a socket needs an adapter and a 4th field
  - a level is held, an event is drained - one press of ']' is one upshift, not "keep upshifting"
- PART 7 - drive somewhere real. `SCENE = "mcity"` puts the car in the Mcity digital twin
  - road surface as a collision mesh, 860 scenery placements over 230 meshes, `MCITY_DETAIL` trades scenery for frame rate
- PART 8 - control something that isn't a car. `PLANT = "rover"` or `"crane"`
  - the crane has no Chrono::Vehicle at all, and the payload swings with nothing to damp it but the operator
- All three ride the same loop, the same three numbers, and the same operator console, unchanged

Where this goes next

- Chrono::HIL (github.com/zzhou292/chrono-HIL): the same three-float interface, scaled up
  - SDL steering wheels with force feedback, cumulative real-time timer, multi-vehicle traffic
  - ROS 2 and Unity bridges, teleoperation over the network, deformable (SCM) terrain
- Chrono::Sensor: cameras/lidar rendered from the vehicle for the operator's screen
- Chrono::Synchrono: several humans in the same world, each on their own machine
- Takeaways
  - Real time in Chrono is soft: spin once per step, and measure the drift
  - The human interface is three floats per step, sampled once and smoothed
  - Never let the input device block the physics loop
  - Levels are held, events are drained - a gear is an event
  - Nothing in the pattern needs a vehicle: swap the plant, keep the loop

Any questions?

Thank you.

kasha2@wisc.edu
Simulation-Based Engineering Lab
University of Wisconsin - Madison

Lab website: http://sbel.wisc.edu
Chrono website: http://www.projectchrono.org
Source code: https://github.com/projectchrono
Tutorial code + slides: https://github.com/ksha23/chrono-hil-tutorial